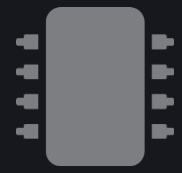
CSI101

Computer Systems







Lecture 1 — Introduction to Computer Systems

Dr. Kok Hoe WONG

Overview

1. Course info
2. Overview of computer systems
3. Hardware and software
4. Backward (downward) compatibility
5. VHDL
6. Hierarchy of systems
7. Trends of computing
8. Input-Process-Output model
9. Machine instructions
10. The von Neumann model
11. Harvard architecture

1 Course information

Aims, learning outcomes, syllabus and reading.



Teaching Staff

Meet the instructors and support team for this module.

Module Leader

Dr. DaWei LIU

SD455 · dawei.liu@xjtlu.edu.cn

Co-Teacher

Dr. Kok Hoe WONG

SD431 · kh.wong@xjtlu.edu.cn

Teaching Assistant (TA) Support

- There will be around 10 TAs to support this module by providing on-site consultations via Help Session bookings.
- They will also be in the labs to support you during your hands-on assembly programming practice.

Course Assessment

Two components make up your final grade for this module.

30%

Class Test

- Held 7-8 PM on November 20th (Friday)
- An online quiz, assessed concurrently in several computer labs
- Conducted via the Safe Exam Browser (SEB)

70%

Final Exam

- Date to be confirmed later
- Multiple choice questions
- Fill-in-the-Blank questions
- Assembly programming questions

100% of your final grade — plan revision time for both.

Tutorials and Labs

Code demo is conducted during tutorials followed by hands-on practice in the labs the following week.



Tutorials

- Four one-hour sessions
- Held in Weeks 3, 4, 5, and 6
- Please check your timetable for your correct group



Labs

- Four two-hour sessions in the computer labs
- Held in Weeks 4, 5, 6, and 7
- Tools: Visual Studio, Visual C++, Inline Assembler
- Practise exercises covered in the tutorials the week before

Stuck on something? Book a Help Session with a TA.

Booking of Help Sessions

Students can book 60-minute sessions with a Teaching Assistant (TA).

A Book at least 2 working days ahead — this gives your TA time to prepare.

B State your issue or question clearly so the TA can prepare to help you.

C Meet must be held on campus — the default spot is SC221, or message your TA to change it.

D Wait for your TA to confirm before heading to the session.

To book your Help Session, click here: [Book Your Help Session with TA | LM Core]

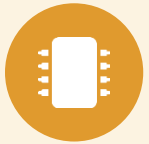
What is this course about?

You already write programs. This module asks what happens when you run them.



Java — the high level

- How programs are written for people to read
- Classes, objects, methods, libraries
- Correctness and design are the concerns
- The machine is deliberately hidden from you



Computer Systems — the low level

- What happens inside the machine as it runs
- How the hardware is organised, and why
- How programs are written at the lowest level
- Where performance comes from — and where it goes

Same program. Two very different questions.

Aims of the module

To give you a working understanding of computer systems — organisation, operating systems, and low-level programming — as the foundation for later years.



Understand

Build a comprehensive picture of computer systems, including computer organisation and operating systems.



Apply

Design and write assembly programs that solve simple problems, working directly with the instruction set.



Progress

Establish the foundation for later-year modules in computer science and computer systems.

Learning outcomes

By the end of the module, you should be able to:

A Identify and explain the core components of computer systems, their functions, and how they interact.

B Distinguish between the key architectural levels of a computer system and explain how their functions differ.

C Implement simple algorithms in assembly language to solve fundamental problems.

D Describe the core functions of a modern operating system: managing hardware resources and serving user processes.

Syllabus

01
Introduction

02
Components: Processors

03
Components: Primary Memory

04
Components: Secondary Memory and Peripherals

05
Architecture: The Digital Logic Level

06
Architecture: The Machine Program Levels

07
Memory System Organisation and Architecture

08
Interfacing and Communication

09
Assembly Programming

Reading and resources

You do not need to buy everything. Use one core text and dip into the others.



Core text

- Tanenbaum, Structured Computer Organization — the [Tan01] quotations in these slides
- Read alongside the lecture, not after it



Wider reading

- Comer, Essentials of Computer Architecture — strong foundation in computer architecture and organization



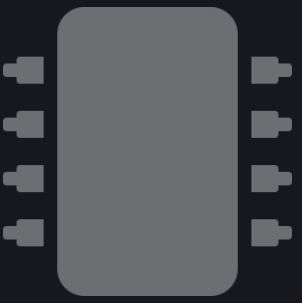
Practice

- The inline assembler on Visual Studio IDE
- Hands-on practical exercises on assembly language in the labs

2

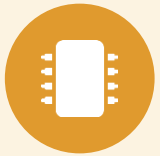
Overview of computer systems

What a computer system is, the shapes it comes in, and how it got here.



What is a computer system?

A combination of hardware, software and data, organised so that it accepts input, processes it under the control of a stored program, and produces useful output.



Hardware

The physical machine. Processor, memory, storage, and the devices that connect it to the world.



Software

The instructions. System software drives the hardware; application software serves the user.



Data

What is being manipulated. Held in the same memory as the program, in the same binary form.

Types of computers

Physical size falls, performance rises, and the number of computers per person keeps climbing.

1960s

Mainframes

Large organisations, high-throughput transaction work

1970s

Supercomputers

Scientific computation at the limit of what is buildable

1980s

Workstations

Single-user engineering and design machines

1980s

Personal computers

Microcomputers that put a machine on every desk

1980s

Microcontrollers

Embedded: calculators, appliances, cars, sensors

1980s

Servers

Network, web and file services for many clients

2000s

Cloud

Computing rented by the hour from vast data centres

Today

System-on-chip

Whole computers on a die: phones, watches, earbuds

Downsizing →

↑ Increasing performance

Computer generations: the first three

Each generation is defined by the switching technology at its heart.

1 First generation

1944 – 1958

SWITCHING TECHNOLOGY

Vacuum tube

MACHINES

Mark I, ENIAC, UNIVAC

Room-sized, tens of kilowatts, tubes failing daily



2 Second generation

1959 – 1963

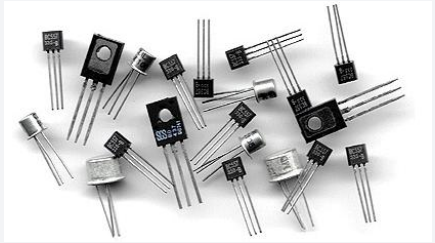
SWITCHING TECHNOLOGY

Transistor

MACHINES

IBM 1401, IBM 1410 with 1402 card reader/punch

Smaller, cooler, far more reliable; business computing arrives



3 Third generation

1964 – 1970

SWITCHING TECHNOLOGY

Integrated circuit

MACHINES

IBM System/360, DEC PDP series

Families of compatible machines — software outlives the hardware



The fourth generation, and what follows

FOURTH GENERATION · 1971 – NOW

LSI and VLSI

- Large- and very-large-scale integrated circuits put a processor on one chip
- Word processing, spreadsheets, databases and graphics reach ordinary users
- Cray-1, Apple II, IBM PC, laptop, palmtop, smartphone, tablet

EMERGING · TODAY

Parallel and heterogeneous

- Many cores rather than one fast core
- Specialised units alongside the CPU: GPU, neural and signal processors
- Chiplets, 3D stacking, and computing rented at cloud scale

2,300

transistors on the Intel 4004, 1971

→

over 100 billion

transistors in a single high-end package today

3

Hardware and software

Two halves of one subject — and why studying either alone goes wrong.



Why study both together?

Courses used to teach one or the other. The specialisation caused real, expensive problems.



Unsuitable equipment is ordered

Specifications written by people who cannot predict how the machine will actually be used.



Software fails to exploit the hardware

New circuits offer real performance advantages that the software never asks for.



Nobody can diagnose the problem

When a system is slow, neither side can tell whether the fault is above or below the line.

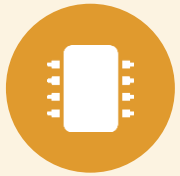
Hardware sets the ceiling. Software decides how close you get to it.

Computer hardware: five categories



Input

Keyboard, mouse, touchscreen, scanner, microphone, sensors



Processing

CPU, and increasingly GPU and specialised accelerators



Output

Display, printer, speaker, actuators



Storage

Main memory, SSD, hard disk, optical and removable media

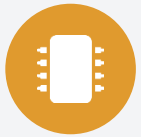


Communications

Network interfaces, wireless radios, and the buses inside the box

Inside the box

The most visible part of a computer system is the hardware — it is physical, you can touch it.



Central Processing Unit

- The active part of the machine
- Performs calculations, comparisons and control
- Fetches and executes instructions one after another



Main memory (RAM)

- Primary or working storage
- Holds programs and data for immediate access by the CPU
- Volatile — the contents vanish when power is removed



Secondary storage

- Long-term storage that survives power-off
- Holds programs and data until they are needed
- SSD, hard disk, optical and removable media



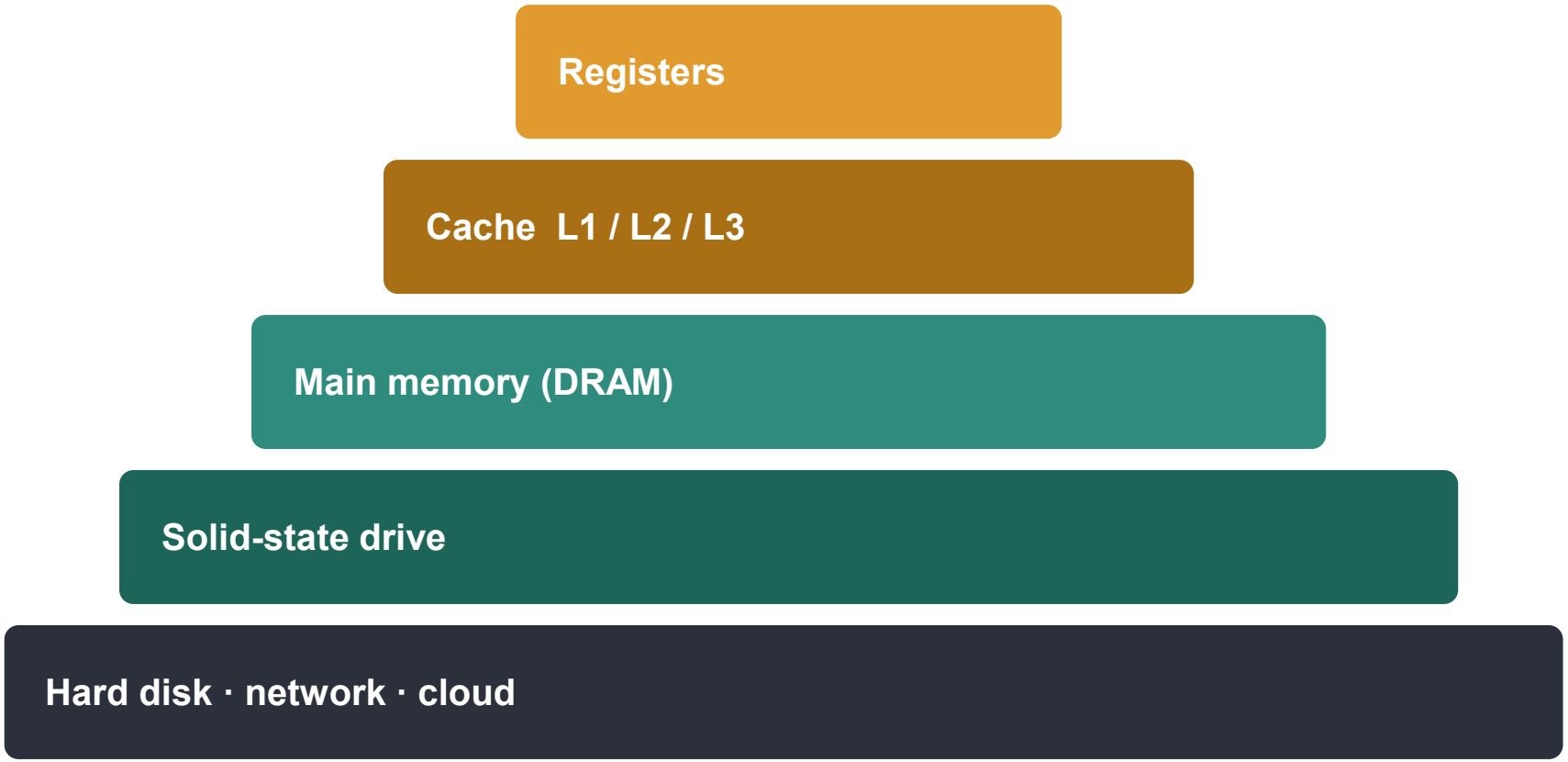
Input and output devices

- Input: keyboard, mouse, scanner, sensors
- Output: monitor, speaker, printer
- The system's only contact with the outside world

Memory is volatile — which is why every program you write eventually has to talk to storage.

The memory hierarchy

No single memory technology is fast, large and cheap at once. Systems use several, arranged in layers.



	CAPACITY	ACCESS TIME
Registers	~1 KB	**under 1 ns**
Cache L1 / L2 / L3	KB – tens of MB	**1 – 30 ns**
Main memory (DRAM)	gigabytes	**50 – 100 ns**
Solid-state drive	hundreds of GB – TB	**50 – 100 µs**
Hard disk · network · cloud	TB and beyond	**5 – 10 ms**

Faster means smaller and dearer per byte. The hierarchy is how a system gets speed and capacity at the same time.

Computer software: two families



System software

- Communicates directly with the hardware
- Manages the machine’s resources: processor time, memory, devices
- Lets application programs work together without knowing about each other
- Operating systems, device drivers, firmware, utilities

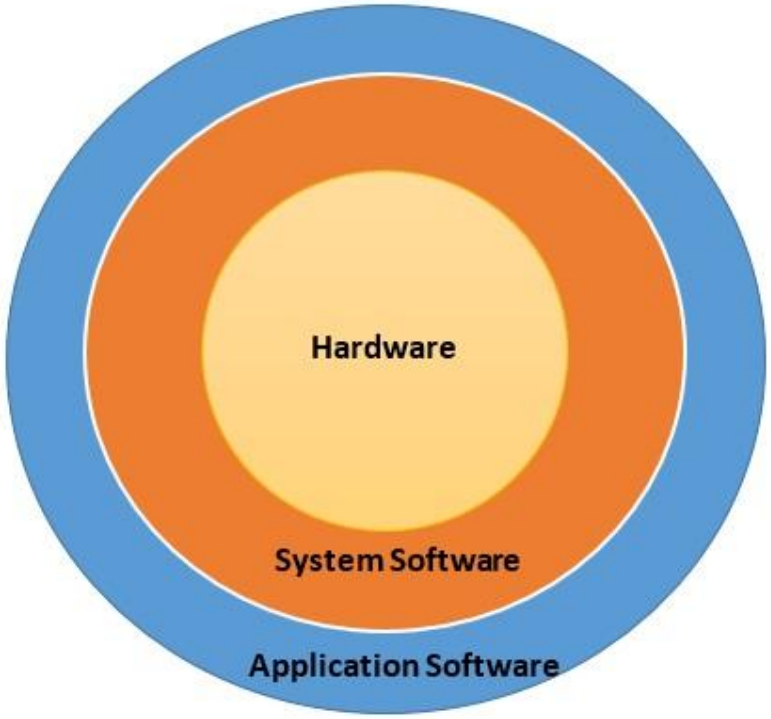


Application software

- Does something directly useful for the user
- Assumes the hardware has already been tamed by system software
- Written against the interfaces the system software provides
- Browsers, spreadsheets, editors, games, engineering tools

Hardware can only do very simple things — but very quickly. Software is how we combine those simple steps into something useful.

Hardware and software together



“Hardware and software are logically equivalent.” **[Tan01]**

Hardware evolution: Moore’s Law

Gordon Moore



- Born 3 January 1929; died 2023
- Co-founder of Intel Corporation
- Made the observation in 1965 and revised it in 1975

THE OBSERVATION

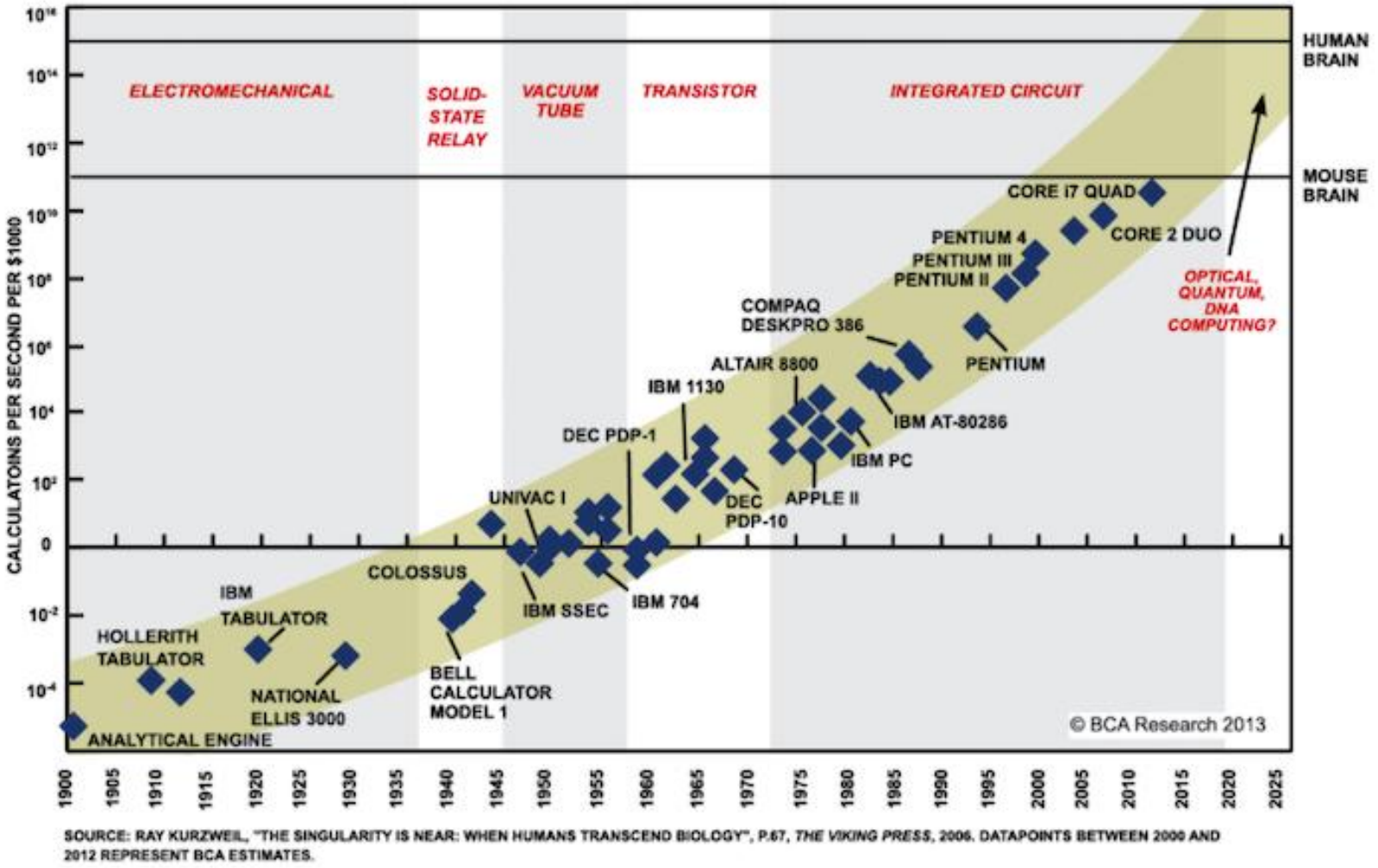
The number of transistors that can be placed on a chip of a given area roughly doubles every two years.

- So, a circuit designed two years ago now fits in half the area
- Often quoted as every 18 months; Intel’s own statement says two years

It is an observation and an industry target — not a law of physics.

The industry planned around it for fifty years, which is part of why it held for so long.

Moore's Law: the evidence



SOURCE: RAY KURZWEIL, "THE SINGULARITY IS NEAR: WHEN HUMANS TRANSCEND BIOLOGY", P.67, *THE VIKING PRESS*, 2006. DATAPOINTS BETWEEN 2000 AND 2012 REPRESENT BCA ESTIMATES.

A straight line on a log scale means exponential growth.

≈ 50 million ×

more transistors on a chip in 2022 than in 1971.

Moore’s Law today

Transistor counts still rise. What stopped rising is clock speed — and that changed how we program.

Dennard scaling ended

Around 2005 power density stopped falling with each shrink. Clock speeds levelled off at roughly 3-5 GHz and have stayed there.

More cores, not faster cores

Extra transistors went into parallelism: multiple cores, wider vector units, larger caches.

Specialised silicon

Graphics, signal and neural processors do specific jobs far more efficiently than a general-purpose core.

The economics got harder

Each new process node costs more to develop and to build. Progress continues, but not for free.

Performance now comes from parallelism — and from software that understands the hardware beneath it.

When hardware and software push each other



Windowing interfaces — WIMP

- Windows, icons, menus, pointer
- A by-product of the microprocessor revolution, which put fast bitmapped graphics on every desk
- The hardware made the interface possible; the interface then made graphics hardware essential
- Today the same loop runs through touch, voice and gesture



The Internet — connecting the networks

- A research network joining separate networks together
- The launch of the Netscape browser turned it into a mass medium
- Demand for pages, images and video then drove network and graphics hardware hard
- Streaming and video calls are the current round of the same pressure

Neither side leads for long.

Backward (downward) compatibility

Most software written for computers with older hardware will still run on computers with newer hardware.

WHY DO WE BOTHER?

It protects what people already own

A business may have far more invested in its software and data than in the machine running them.

It makes upgrading safe

Buying a faster machine stops being a risk, so people buy more often.

It grows the platform

More software attracts more buyers, which attracts more software. Compatibility keeps that loop turning.

Example: code compiled for the Intel 8086 in 1978 still runs on an x86 processor bought today.

Compatibility: what it costs

Every promise kept to old software is a constraint on new hardware.

THE PRICE PAID

- Old instructions and operating modes must be kept, and tested, forever
- Silicon and design effort go into decoding formats almost nobody uses
- Early design decisions become permanent, however awkward they turn out to be
- Verification cost grows with every generation of accumulated behaviour

THE ALTERNATIVE

- Break compatibility, and bridge the gap with translation or emulation
- Apple has done this three times: 68k → PowerPC → x86 → ARM
- Each move came with a translation layer, so old software kept working, slowly, for a while
- Cleaner hardware, but a hard deadline for everyone who writes software for it

Compatibility is a business decision as much as an engineering one.

VHDL: describing hardware in code

VHDL Very High-Speed Integrated Circuit Hardware Description Language

It narrows the split

The gap between people who think about hardware and people who think about software has been wide for decades. Hardware description languages narrow it.

Structure and behaviour

One language specifies both what a circuit is made of and what it does — so a design can be written the way a program is written.

Simulation before silicon

A design can be run and tested as a simulation long before anything is manufactured, when mistakes are still cheap.

Straight into layout

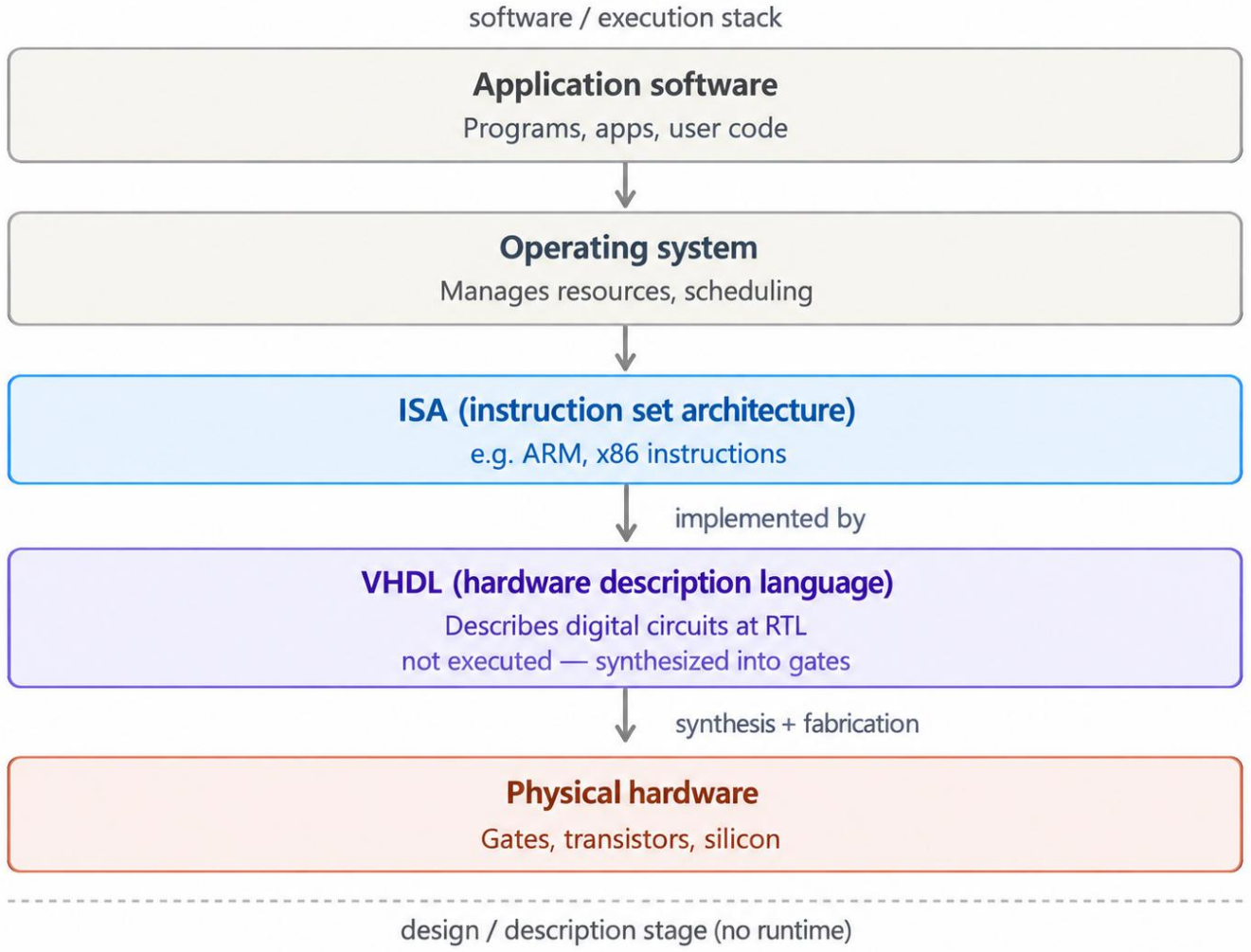
The description feeds automatic synthesis and layout tools that arrange the final circuit.

Hardware is now designed by writing, testing and version-controlling code.

VHDL: visualizing microarchitectural position

What actually is VHDL?

VHDL a step before hardware exists physically. It is a hardware description language used to design and simulate digital circuits, which are then synthesized into gates and fabricated into silicon. So, it bridges the gap between architectural design and physical hardware, rather than sitting between software and hardware in an execution sense.



VHDL: what the code looks like

```
entity half_adder is
  port ( a, b   : in  std_logic;
         sum    : out std_logic;
         carry  : out std_logic );
end half_adder;

architecture behaviour of half_adder is
begin
  sum   <= a xor b;
  carry <= a and b;
end behaviour;
```

READING IT

- entity — the outside view: what wires go in and out
- architecture — the inside view: what the circuit does
- <= assigns a value to a signal, continuously, not once
- This is a half adder: two bits in, a sum bit and a carry bit out
- Written like software; synthesised into gates

You will meet this properly at the digital logic level, later in the module.

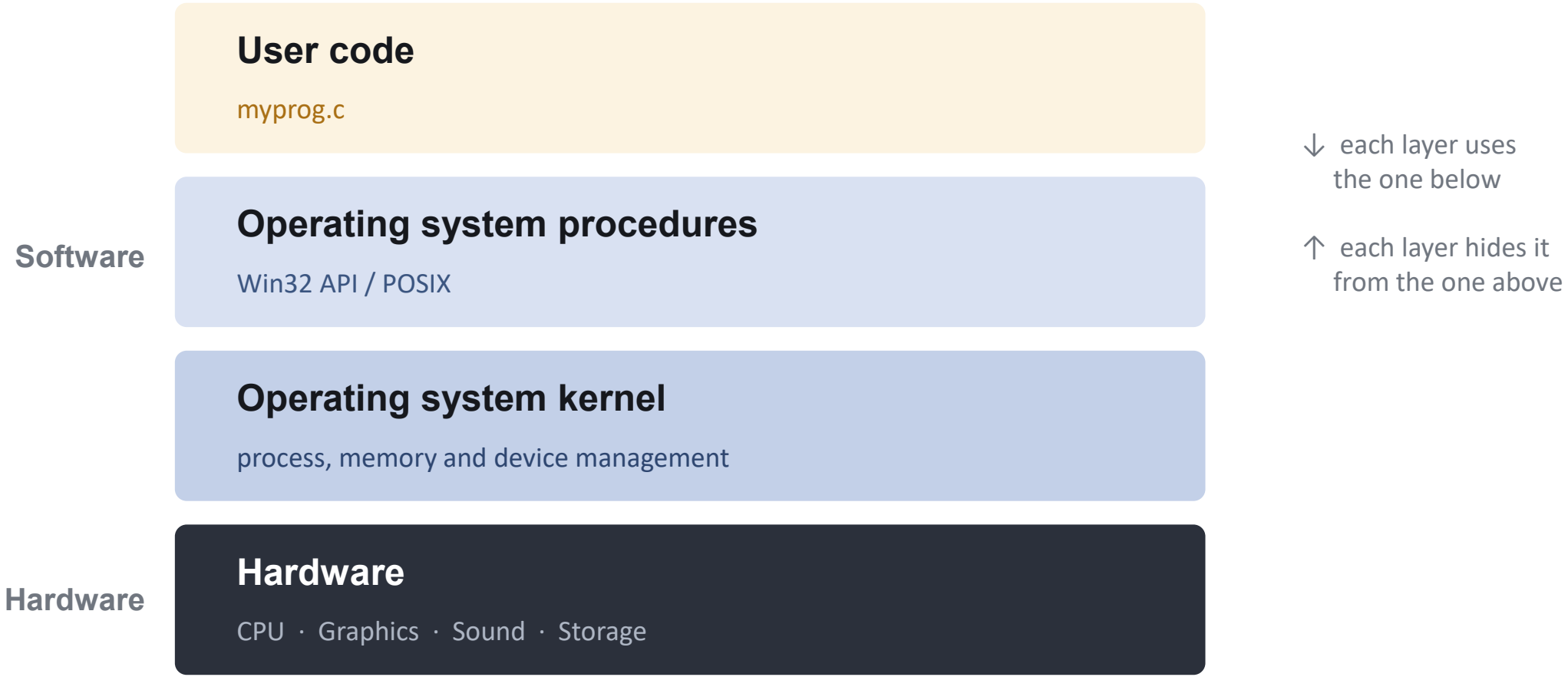
4

Hierarchy, the operating system and trends

Why computers are built in layers, and what sits on top of them.



Computer systems as a hierarchy



"… computer systems can be designed in a systematic, organized way… calling this approach structured computer organization." [Tan01]

Why hierarchy works



Each level hides the mess below it

A level offers a simpler interface upward than the one it received from beneath.



You can be productive at one level

Writing a useful program does not require knowing how a transistor switches.



Levels can evolve independently

A new graphics card does not break your Java program, because they never met directly.



Faults can be isolated

Each level can be tested against its own interface, so a problem can be located rather than guessed at.



The same program runs on different machines

As long as the interface holds, what is underneath can be replaced entirely.



Complexity becomes manageable

No one person or team has to hold the whole system in their head at once.

What the operating system gives us

It brings the hardware's functions out	Capabilities buried in the hardware are surfaced by the operating system and offered to the user.
It stands between programs and hardware	Your programs reach the hardware through the functions the operating system provides — never directly.
Ease of programming	One interface to learn instead of a separate one for every device ever made.
Protection	For the system itself, and for each user against the mistakes of the others.
Fairness and efficiency	Processor time, memory and devices are shared out rather than seized by whoever asks first.

Trends of computing

Each era is defined by what the computing was for.

Scientific	Business	Personal	Pervasive	Mobile
computation	**data**	**interaction**	**ubiquity**	**mobility**
Calculating what could not be calculated by hand	Recording, storing and reporting on transactions	One machine, one user, a screen and a pointer	Computers embedded in objects, mostly invisible	Computing that travels with the person, always connected

What is next?

Every era on this slide looked obvious only in hindsight.

What comes next?

Candidates, not predictions — and several will turn out to be the same trend.



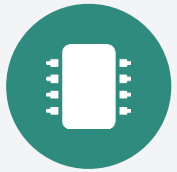
Intelligent computing

Machine learning models running everywhere, with dedicated accelerators in phones, cars and data centres.



Edge computing

Processing data where it is created rather than shipping it to a distant data centre first.



Heterogeneous computing

The right kind of core for each job — general, graphics, signal, neural — on one package.



Sustainable computing

Performance per watt becoming the headline number, because energy is the binding constraint.



Quantum computing

Still research, and useful only for a narrow class of problems — but that class matters.

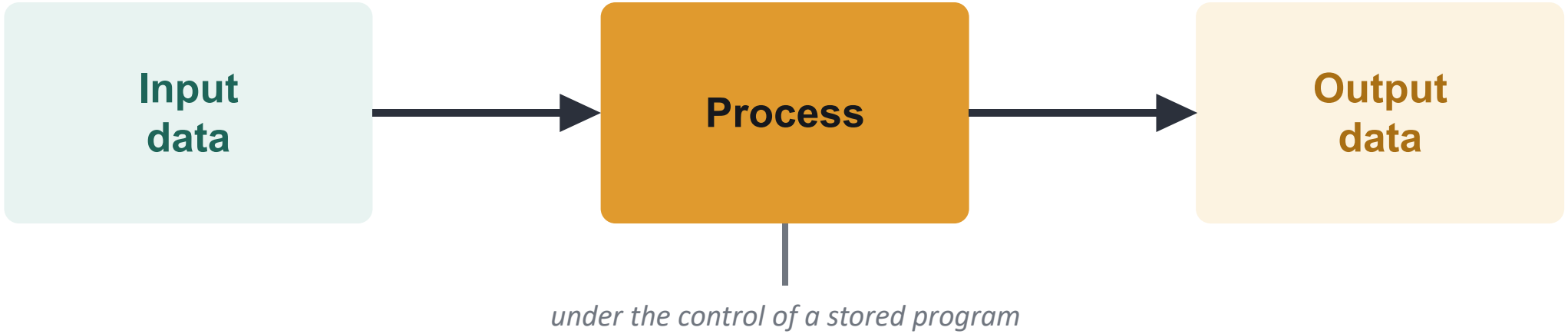
5

Models of computation

Input-Process-Output, machine instructions, and the two architectures that organise them.



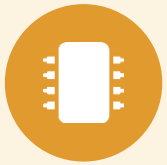
The Input-Process-Output model



- The Input-Process-Output model is the fundamental structure of the current generation of digital computers
- The process is controlled by a special, custom-made program — which is what makes one machine able to do many jobs
- This was the essential scheme of the von Neumann model, which we come to shortly
- It is a highly conceptualised model: useful for reasoning, deliberately silent about how any of it is built

Three components of a computer system

Implementing the Input–Process–Output and von Neumann models needs all three, together.



Hardware

- The physical machine you can touch
- Processor, memory, storage, devices
- Fast, but capable only of very simple operations



Software

- The instructions that direct the hardware
- System software and application software
- Turns simple operations into useful work



Data

- What is actually being manipulated
- Held in memory in the same binary form as the program
- Meaningless until something interprets it

Remove any one of the three and there is no system — only parts.

Machine instructions

- The CPU's whole job is to execute machine instructions
- Every CPU has its own instruction set — typically 100 to 200 instructions
- For a particular machine, that set is fixed and cannot be extended
- Different CPUs have broadly similar instruction sets, but there is no standard one
- An instruction is only a binary pattern; assembly language gives each pattern a name a human can read

ONE INSTRUCTION, THREE VIEWS

Binary	`1000 1001 1101 1000`
Hexadecimal	`89 D8`
Assembly	`MOV EAX, EBX`

All three say the same thing to the processor. Only the last one says it to you.

You will write assembly for one specific machine in this module — and that is the point. There is no generic CPU.

Machine instruction categories

Almost every instruction set, on any processor, divides into these four groups.



Input and output

`IN, OUT`

Present on Intel x86; many other processors have no I/O instructions at all and use memory-mapped addresses instead.



Data transfer and manipulation

`MOV, ADD, MUL, AND, OR`

Moving values between registers and memory, and doing arithmetic and logic on them. The bulk of any program.



Transfer of program control

`JMP, JC, CALL, RET`

Changing which instruction runs next. This is where loops, conditionals and procedure calls come from.

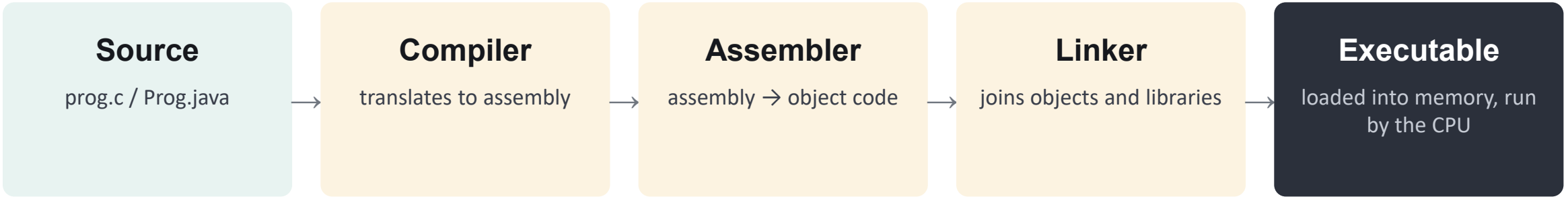


Machine control

`HLT, INT, NOP`

Halting the processor, resetting hardware, raising an interrupt — controlling the machine rather than the calculation.

From high-level language to machine code



WHY TRANSLATE AT ALL?

- The processor understands nothing but binary opcodes
- High-level languages are written for people, not for silicon
- One source file can be compiled for many different machines

SO WHY LEARN ASSEMBLY?

- It is what the machine actually runs — everything else is a convenience
- Some work has no other option: device drivers, boot code, tight embedded systems
- It makes the cost of a high-level construct visible

John von Neumann

1903 – 1957



An Austria-Hungary-born American mathematician, and one of the most broadly influential scientists of the twentieth century.

THE PAPER THAT NAMED AN ARCHITECTURE

First Draft of a Report on the EDVAC
30 June 1945

HE CONTRIBUTED TO

- Quantum physics
- Functional analysis
- Set theory
- Topology
- Game theory and economics
- Computer science
- Numerical analysis
- Hydrodynamics

The von Neumann model

Formulated in the late 1940s: a general-purpose machine, controlled by an executable program.



A program is a list of instructions

Written to direct the machine through a particular task, one step at a time.



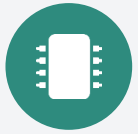
Program and data share one memory

Both are held in the machine's store, and both are represented as binary codes.



That memory can be rewritten

This is what makes the model powerful: the machine can be given a new task without being rebuilt.

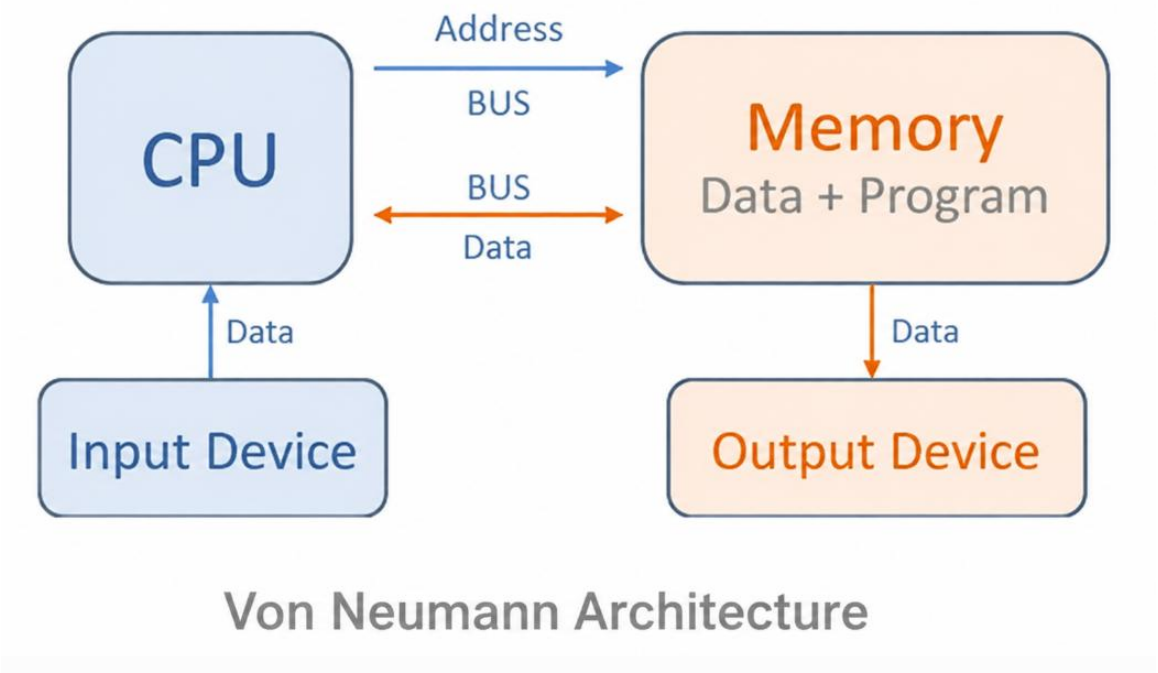


The processor is the active part

It fetches instructions from memory and executes them, in order, unless told otherwise.

How does this specification make the machine ‘general-purpose’?

The model, and its bottleneck



Von Neumann Architecture

THE DEVICES

- The input device carries information from the user into the machine’s memory
- The output device lets the user see the results of the program that has run

THE VON NEUMANN BOTTLENECK

- Instructions and data cross the same path between processor and memory
- So, the CPU is continually forced to wait for what it needs to arrive
- Caches, wider buses and prefetching all exist to soften this one problem

Data or instruction?

THE PROBLEM

Both are binary. So how does the machine tell them apart?

```
0100 0001 0100 0010
```

- As a number: 16,706
- As text: the two characters A and B
- As an instruction: whatever the opcode 0x41 0x42 means on this processor

THE SOLUTION

Nothing in the bits. Everything in the context.

- Data and instructions are both stored in memory, as the model requires
- The CPU knows where the program begins and keeps track of where it is
- Bits fetched as instructions are decoded as instructions; bits fetched as operands are not
- Both must be in a coded form the CPU is built to understand

A stored program is data that the machine has agreed to obey.

Who invented the stored program?

1936

Konrad Zuse

Mentioned the concept in a patent application, years before any of the others wrote it down.

December 1943

Mauchly and Eckert

Wrote about the stored-program concept while working on ENIAC, a general-purpose computing machine.

30 June 1945

John von Neumann

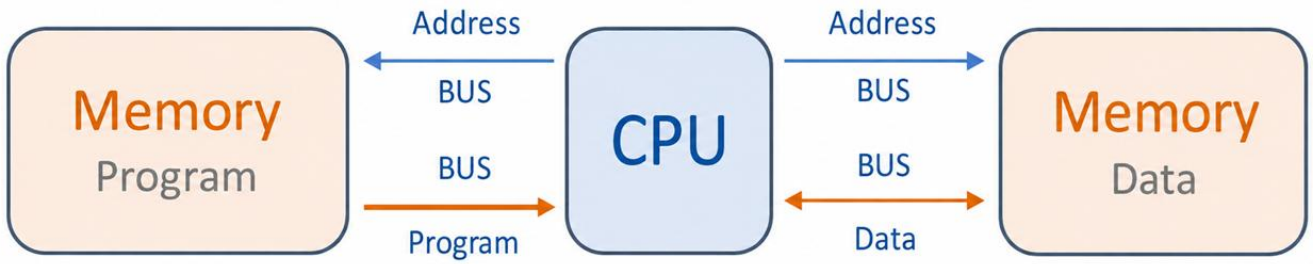
First Draft of a Report on the EDVAC. It circulated widely with only his name on it — and the name stuck.

The architecture carries one name. The idea had at least three parents.

A useful reminder that attribution in computing history is rarely as tidy as the textbook makes it look.

Harvard architecture

A variation that has gained ground: separate the data from the programs.



Harvard Architecture

HOW IT DIFFERS

- Programs and data live in physically separate memories
- Each has its own bus, so an instruction fetch and a data access can happen at the same time
- The intention is a higher transfer rate, and so better throughput

WHERE YOU WILL MEET IT

- Digital signal processors, where predictable throughput matters most
- Microcontrollers such as the Peripheral Interface Controller (PIC) and Advanced Virtual RISC (AVR) families
- Anywhere program code sits in read-only memory and data does not

So, which one won?



von Neumann won

- Nearly all main memory — RAM and drives alike — holds data and instructions together
- The specification has held up for more than seventy years
- It is implemented, in some form, in almost every computer sold today



Harvard won too

- Most desktop CPUs keep a separate instruction cache and data cache
- The control unit is fed from one; operands come from the other
- Embedded processors often use the separation right down to main memory

THE ANSWER

Modern processors are ‘modified Harvard’: split instruction and data caches sitting on top of one unified main memory.

Summary

1. A computer system is hardware, software and data — studied together, because the interaction between them determines performance.
2. Computers are built as hierarchies, each level hiding the complexity of the one below and offering a simpler interface above.
3. Moore’s Law described fifty years of exponential growth; clock speeds have plateaued, so performance now comes from parallelism.
4. The von Neumann model stores program and data in one rewriteable memory — which makes the machine general-purpose, and creates the bottleneck.
5. Harvard architecture separates the two for throughput. Modern processors use both, at different levels.